UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa

Program Studi Sistem Informasi

TEMPLAT LAPORAN

TUGAS 2

Postman Collection dan Automated Test Script

Pengujian *RESTful API* dengan *Postman Automation*

Pengembangan Aplikasi Web II | SUB-CPMK-7

Nama dan NIM	Kirana Cinta Mentari (2428240078)
Kelas	SI5B
Judul project / topik API	RESTfull API Kampus: Alumni
Tautan repositori	https://github.com/kiranacintamentari2428240078/si5b_kuis1_KiranaCintaMentari
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	21 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah laporan **Tugas 2**: menguji RESTful API project akhir Anda dengan Postman. Seluruh isinya menggunakan **API milik Anda sendiri**, bukan API Mahasiswa pada contoh materi. Pelajari langkahnya pada materi Pertemuan 8 sebelum mengisi.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya [sumber-daya], dengan data API Anda.
2. Bukti berupa gambar (*screenshot*) diisi pada kotak bergaris putus-putus. Cuplikan kode test script **ditempel sebagai teks**, bukan gambar.
3. Setiap bukti memiliki keterangan dan kotak **Penjelasan**. Tulis penjelasan dengan kalimat Anda sendiri dan sebut nama *request* atau fungsi dari API Anda.
4. Simpan hasil akhir sebagai PDF, lalu kumpulkan melalui LMS sesuai ketentuan dosen (format nama berkas dan batas waktu mengikuti petunjuk di LMS).

B. Bukti yang diminta

No	Bukti	Bagian	Jenis
1	Daftar request dalam Collection	Bab 2	Screenshot (wajib)
2	Cuplikan test script	Bab 3	Kode teks (wajib)
3	Ringkasan Collection Runner	Bab 4	Screenshot (wajib)
4	Hasil Newman di terminal	Bab 5	Screenshot (pengayaan)

C. Syarat tangkapan layar

Bukti	Yang harus terlihat
Collection	Nama Collection, kedua folder (bila ada), dan seluruh nama request. Buka semua folder sebelum mengambil gambar.
Ringkasan Runner	Nama Collection, Environment yang dipakai, jumlah request, jumlah test **lulus** dan **gagal**. Seluruh test harus lulus (0 gagal).
Newman	Perintah yang diketik, daftar hasil, dan tabel executed dan failed di bagian akhir.
Larangan	Gambar dipotong sehingga angka hasil tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Sebelum mengambil screenshot Runner

Nyalakan ulang server agar data kembali ke kondisi awal, pilih Environment yang benar, lalu jalankan seluruh Collection. Bila ada test gagal, perbaiki dulu dan jalankan ulang sampai 0 gagal.

Bab 1 Identitas API dan Daftar Endpoint

Bab ini menjelaskan API yang diuji, agar pembaca memahami cakupan pengujian pada bab berikutnya.

1.1 Deskripsi singkat API

Penjelasan:

API yang saya buat adalah RESTful API untuk mengelola data alumni dengan topik pendataan alumni perguruan tinggi. Entitas utamanya adalah Alumni, yang memiliki atribut seperti id, nim, nama, prodi, tahun lulus, dan pekerjaan . API ini menyediakan operasi CRUD lengkap, yaitu menampilkan semua data alumni (GET), menampilkan satu alumni berdasarkan id (GET), menambah data baru (POST), mengubah data (PUT), dan menghapus data (DELETE).

Selain operasi normal, API ini dilengkapi mekanisme keamanan dan penanganan error. Request POST dilindungi API key sehingga request tanpa API key ditolak dengan status 401. Data yang tidak lengkap ditolak dengan status 400 beserta pesan field wajib diisi, JSON yang rusak ditangani dengan pesan “Format JSON tidak valid”, dan data yang sudah dihapus mengembalikan status 404 dengan pesan “Data tidak ditemukan”.

Teknologi yang dipakai adalah Node.js dengan framework Express.js yang berjalan di http://localhost:3000, dengan format pertukaran data JSON. Pengujian dilakukan menggunakan Postman, yaitu Collection untuk menyusun request, Environment untuk menyimpan variabel seperti base_url dan AlumniId, serta Runner untuk menjalankan seluruh request dan test secara otomatis.

1.2 Daftar endpoint

Tuliskan **seluruh** *endpoint* API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey atau mekanisme serupa.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/alumni	Tidak	200	Menampilkan semua data alumni.
2	GET	/alumni/:id	Tidak	200	Menampilkan satu data alumni.
3	POST	/alumni	Ya	201	Menampilkan satu data alumni berdasarkan id.

No	Metode	Alamat	API key	Status sukses	Fungsi
4	PUT	/alumni/:id	Ya	200	Mengubah data alumni berdasarkan id.
5	DELETE	/alumni/:id	Ya	204	Menghapus berdasarkan id.

1.3 Variabel Environment

Tuliskan variabel yang Anda simpan di Environment. Jangan menuliskan nilai rahasia sungguhan.

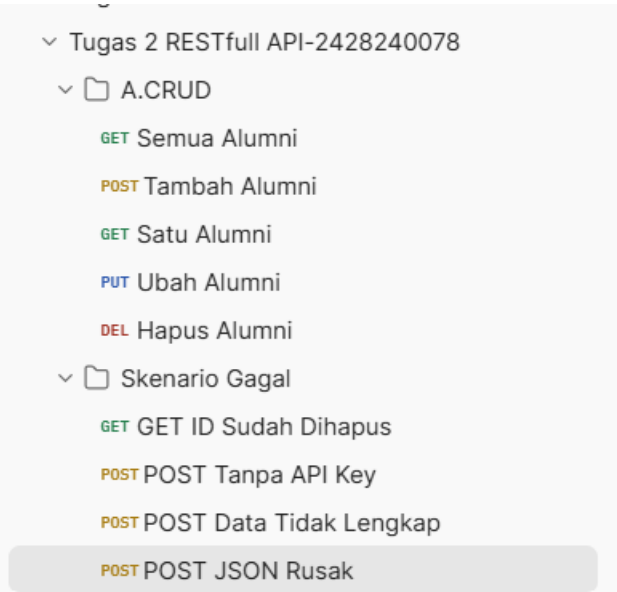
No	Nama variabel	Nilai (contoh)	Fungsi
1	base_url	http://localhost:3000	Alamat server
2	api_key	Si5b_rahasia123	Nilai header kunci API
3	AlumniId	-	ID dari respons POST

Bab 2 Postman Collection

Bab ini membuktikan bahwa seluruh *endpoint* sudah dikelompokkan dalam satu Collection.

2.1 Daftar request dalam Collection

Harus terlihat pada screenshot: nama Collection dan seluruh request di dalam folder. Buka semua folder.



Gambar 1. Collection beserta seluruh request

Penjelasan:

Request dikelompokkan menjadi dua folder berdasarkan tujuan pengujiannya. Folder A. CRUD berisi lima request skenario normal (Semua, Tambah, Satu, Ubah, dan Hapus Alumni) untuk menguji operasi dasar API, sedangkan folder B. Skenario Gagal berisi empat request yang sengaja dikirim dengan kondisi salah (GET ID Sudah Dihapus, POST Tanpa API Key, POST Data Tidak Lengkap, POST JSON Rusak) untuk menguji penanganan error dan validasi. Pemisahan ini memudahkan pembacaan hasil: jika yang gagal ada di folder CRUD, fungsi utama bermasalah, sedangkan jika di Skenario Gagal, penanganan errornya yang bermasalah.

Urutan di folder CRUD mengikuti alur hidup data (Create, Read, Update, Delete). Tambah Alumni harus berjalan lebih dulu karena menyimpan id ke variabel AlumniId yang dipakai Satu, Ubah, dan Hapus Alumni, dan Hapus Alumni diletakkan terakhir karena setelah data terhapus request lain tidak bisa berjalan normal. Folder Skenario Gagal ditaruh setelah CRUD karena GET ID Sudah Dihapus bergantung pada Hapus Alumni agar server benar-benar membalas 404, sementara tiga request POST gagal lainnya tidak bergantung pada data sebelumnya. Dengan begitu, pengujian error baru dilakukan setelah fungsi dasar terbukti berjalan.

2.2 Tabel request

Salin daftar request pada gambar di atas ke tabel berikut. Setiap *endpoint* pada Bab 1 harus punya minimal satu request.

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
1	GET Semua Alumni	GET	{{base_url}}/alumni	A. CRUD
2	POST Tambah Alumni	POST	{{base_url}}/alumni	A. CRUD
3	GET Satu Alumni	GET	{{base_url}}/alumni/{{AlumniId}}	A. CRUD
4	PUT Ubah Alumni	PUT	{{base_url}}/alumni/{{AlumniId}}	A. CRUD
5	DELETE Hapus Alumni	DELETE	{{base_url}}/alumni/{{AlumniId}}	A. CRUD
6	GET ID Sudah Dihapus	GET	{{base_url}}/alumni/{{AlumniId}}	B. Skenario Gagal
7	POST Tanpa API Key	POST	{{base_url}}/alumni	B. Skenario Gagal
8	POST Data Tidak Lengkap	POST	{{base_url}}/alumni	B. Skenario Gagal
9	POST JSON Rusak	POST	{{base_url}}/alumni	B. Skenario Gagal

Bab 3 Automated Test Script

Bab ini menunjukkan *assertion* yang Anda tulis. Setiap request wajib memiliki minimal satu *assertion* status code dan satu *assertion* struktur atau isi data JSON.

3.1 Rekap assertion per request

No	Nama request	Assertion status code	Assertion struktur atau isi JSON
1	GET Semua Alumni	Status code Adalah 200	Response berupa array; setiap data punya id dan nama; response time di bawah 500 ms; data pertama bernama Farhan Akbar.
2	POST Tambah Alumni	Status code Adalah 201	Data baru punya id dan nama sesuai kiriman
3	GET Satu Alumni	Status code Adalah 200	Data yang diambil adalah Axcel.
4	PUT Ubah Alumni	Status code Adalah 200	Prodi berubah menjadi “Teknik Komputer”, dan nama tetap “Axcel”.
5	DELETE Hapus Alumni	Status code Adalah 204	Response tidak memiliki isi (body kosong).
6	GET ID Sudah Dihapus	Status code Adalah 404	Field message bernilai “Data tidak ditemukan”.
7	POST Tanpa API Key	Status code Adalah 401	Field message bernilai “API key tidak valid”
8	POST DATA Tidak Lengkap	Status code Adalah 400	Field message mengandung teks “wajib diisi”.
9	POST JSON Rusak	Status code Adalah 400	Response bertipe JSON (bukan HTML) dan field message bernilai “Format JSON tidak valid”.

3.2 Cuplikan 1: request GET Semua Alumni

Tempel test script **satu request GET** pilihan Anda.

```
pm.test('Status code adalah 200', function () {
  pm.response.to.have.status(200);
});

pm.test('Response berupa array', function () {
  const jsonData = pm.response.json();
  pm.expect(jsonData).to.be.an('array');
});

pm.test('Setiap data memiliki id dan nama', function () {
  const jsonData = pm.response.json();
  jsonData.forEach((item) => {
    pm.expect(item).to.have.property('id');
    pm.expect(item).to.have.property('nama');
  });
});
```

Penjelasan:

Script Post-response pada request GET Semua Alumni berisi tiga pm.test. Test pertama, “Status code adalah 200”, memeriksa bahwa server membalas dengan status 200 OK. Ini penting karena status code adalah indikator paling dasar bahwa request berhasil diproses, sehingga jika terjadi error (500, 404, atau 401) masalah langsung terdeteksi. Test kedua, “Response berupa array”, memeriksa bahwa body response bertipe array, sebab endpoint ini memang harus mengembalikan daftar data; jika tipenya berubah, aplikasi pengguna API seperti frontend bisa error, dan test ini juga menjadi syarat bagi pengecekan berikutnya. Test ketiga, “Setiap data memiliki id dan nama”, memeriksa setiap elemen array agar memiliki properti id dan nama, untuk menjamin struktur data konsisten di seluruh isi array, karena id dibutuhkan sebagai pengenal unik pada request GET Satu, PUT, dan DELETE, sedangkan nama adalah data utama alumni. Ketiganya tersusun berjenjang dari status, tipe data, hingga isi detail, sehingga jika ada yang gagal, tingkat masalahnya mudah diketahui.

3.3 Cuplikan 2: request POST Tambah Alumni

Tempel test script **request POST** (atau request lain yang menyimpan variabel ke Environment).

```
pm.test('Status code adalah 201', function () {
  pm.response.to.have.status(201);
});

pm.test('Data baru memiliki id dan nama sesuai kiriman', function () {
  const jsonData = pm.response.json();
  pm.expect(jsonData).to.have.property('id');
  pm.expect(jsonData.nama).to.eql('Axcel');
});

if (pm.response.code === 201) {
  pm.environment.set('AlumniId', pm.response.json().id);
}
```

Penjelasan:

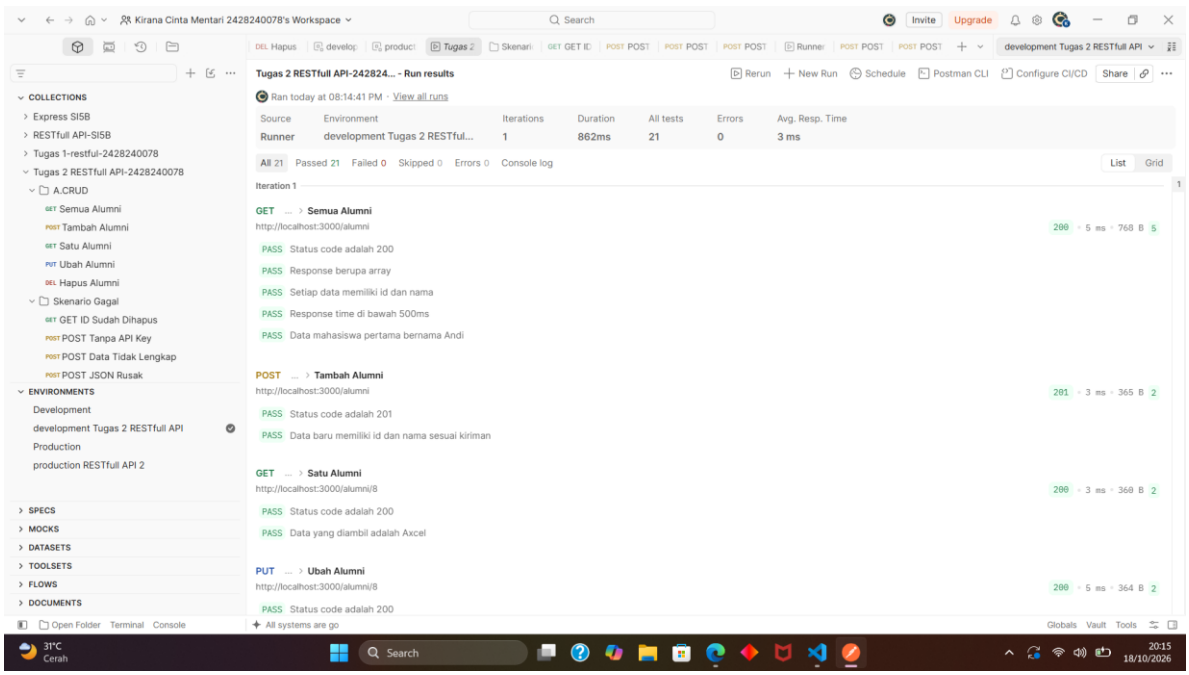
Pada request POST Tambah Alumni, setelah server membalas status 201, script menyimpan id dari body respons ke variabel environment AlumniId lewat pm.environment.set('AlumniId', pm.response.json().id), dan penyimpanan hanya dilakukan jika status code 201 agar variabel tidak terisi nilai yang salah. Variabel ini kemudian dipakai oleh request GET Satu Alumni, PUT Ubah Alumni, DELETE Hapus Alumni, dan GET ID Sudah Dihapus (biasanya lewat URL /alumni/{{AlumniId}}), sehingga semuanya bekerja pada data yang baru dibuat. Jika urutan request ditukar, misalnya GET Satu, PUT, atau DELETE dijalankan sebelum POST, AlumniId masih kosong atau berisi id lama, sehingga server membalas 404 dan test yang mengharapkan 200 atau 204 gagal; demikian pula GET ID Sudah Dihapus yang dijalankan sebelum DELETE akan menerima 200, bukan 404.

Bab 4 Hasil Collection Runner

Bab ini adalah bukti utama bahwa seluruh test lolos saat Collection dijalankan sekaligus.

4.1 Ringkasan Collection Runner

Harus terlihat pada screenshot: nama Collection, Environment, jumlah request, jumlah test lulus dan gagal. Seluruh test harus lulus.



Gambar 2. Ringkasan Collection Runner

Penjelasan:

Pengujian dilakukan dengan menjalankan collection Tugas 2 RESTfull API-2428240078 melalui runner postman (environtment development Tugas 2 RESTfull API). Sebelum runner dijalankan, server dinyalakan ulang agar API berjalan di http://localhost:3000 dengan data awal.

Collection berisi 9 request: 5 CRUD (GET Semua, POST, GET Satu, PUT, DELETE alumni) dan 4 skenario gagal (ID sudah dihapus, tanpa API Key, data tidak lengkap, JSON rusak). Dari request tersebut dijalankan 21 test dengan hasil 21 passed, 0 failed, 0 skipped, 0 error, dengan durasi total 862 ms dan rata-rata respons 3ms.

Contoh test yang lulus: GET Semua Alumni (status 200, response array, setiap data punya id dan nama, response time < 500 ms), POST Tambah Alumni (status 201, data baru sesuai kiriman), GET Satu Alumni (status 200, data sesuai), dan PUT Ubah Alumni (status 200).

4.2 Rekap angka hasil

Nama Collection	Tugas 2 RESTfull API-2428240078
Environment yang dipakai	Development Tugas 2 Restfull API, production RESTfull API 2
Jumlah request dijalankan	9 Request
Jumlah test lulus	21 passed
Jumlah test gagal	0 failed
Tanggal dan jam pengujian	18 Oktober 2026 pukul 20.15 WIB

4.3 Temuan selama pengujian

Catat kegagalan yang pernah muncul sebelum akhirnya lulus. Bagian ini menunjukkan bahwa pengujian benar-benar membantu menemukan masalah. Bila tidak ada, tulis "tidak ada kegagalan".

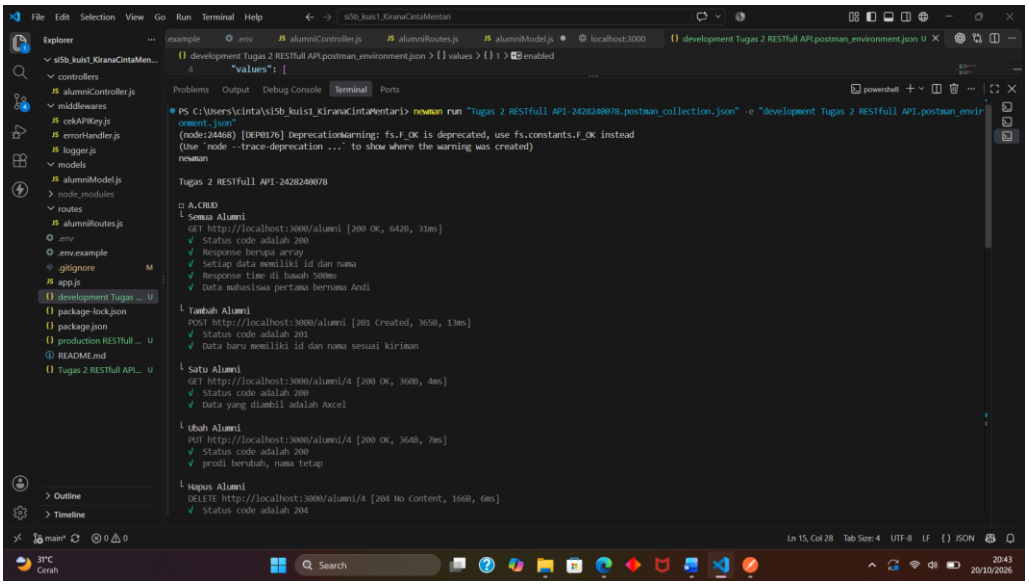
No	Request	Kegagalan (pesan)	Penyebab	Perbaikan
1	POST Data Tidak Lengkap	Error: runtime: request url is empty	Postman Runner tidak menemukan URL untuk request tersebut	Mengisi URL pada request POST yang kosong dengan {{base_ur}}/alumni

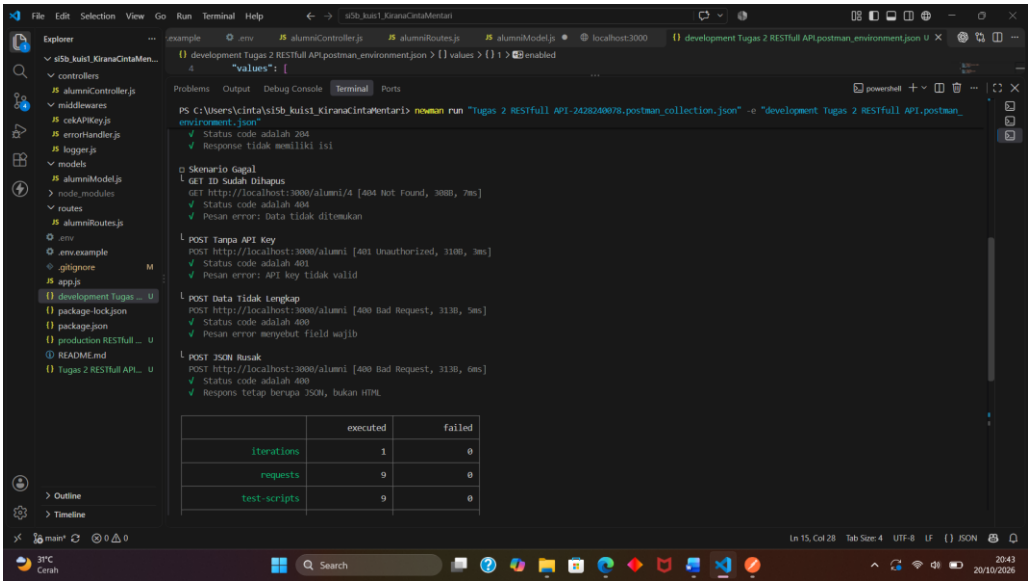
Bab 5 Newman di Terminal (Pengayaan)

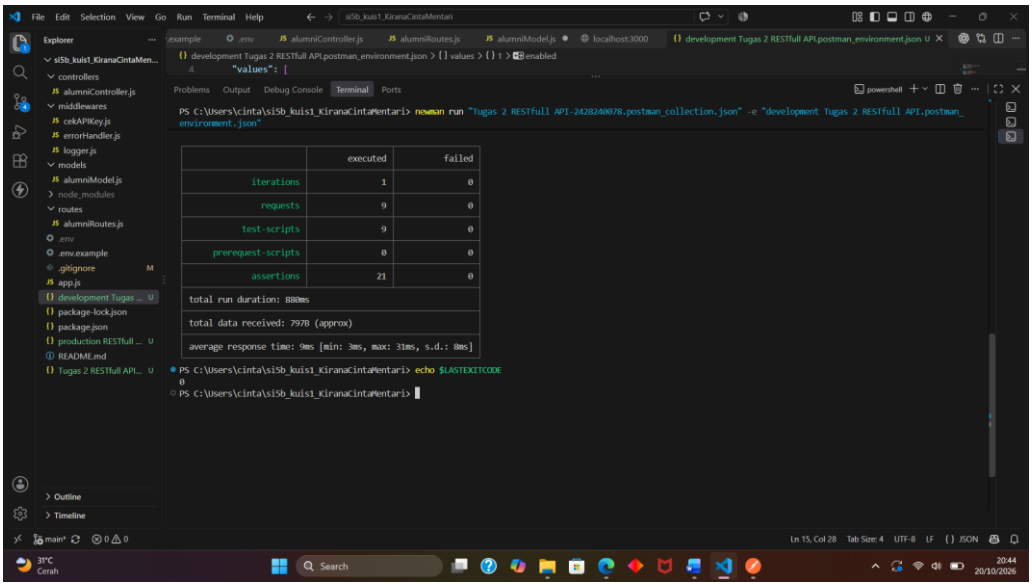
> **Bagian ini opsional**
>
> Bab ini adalah pengayaan dan tidak mengubah kriteria penilaian. Isi bila Anda menjalankan Collection dengan Newman. Bila tidak, tulis "tidak dikerjakan" di bawah judul ini.

5.1 Hasil Newman

Harus terlihat pada screenshot: perintah `newman run ...`, daftar hasil, dan tabel `executed` dan `failed`.







Gambar 3. Hasil menjalankan Collection dengan Newman

Penjelasan:

Collection yang sudah di export dan dimasukkan ke dalam folder tugas dijalankan melalui terminal VS Code dengan perintah berikut, setelah server API dinyalakan http://localhost:3000:

newman run "Tugas 2 RESTfull API-2428240078.postman_collection.json" -e "development Tugas 2 RESTfull API.postman_environment.json"

Perintah -e dipakai untuk membuat file environment yang berisi variable base_url, x-api-key, dan alumniId. Hasilnya, seluruh 9 request dan 21 assertion berhasil dieksekusi dengan 0 gagal (failed = 0), dalam waktu total 880 ms. Setelah Newman selesai, perintah echo $LASTEXITCODE menampilkan nilai 0.

Exit code 0 berarti seluruh test lulus dan proses berakhir dengan sukses. Sebaliknya, exit code 1 berarti ada minimal satu test yang gagal atau terjadi error saat menjalankan collection. Nilai ini penting karena dipakai sistem otomatis (misalnya pipeline CI/CD) untuk menentukan apakah pengujian dianggap berhasil atau tidak.

Bab 6 Repositori dan Berkas

Tautan repositori	https://github.com/kiranacintamentari2428240078/tugas1-restful-2428240078
Lokasi berkas Collection (.json)	development Tugas 2 RESTfull API.postman_environment.json (Pada root repositori)
Lokasi berkas Environment (.json)	development Tugas 2 RESTfull API.postman_environment.json dan production RESTfull API 2.postman_environment.json
Berkas Environment tidak memuat kunci rahasia sungguhan	[✓] Ya
Repositori dapat dibuka dosen	[✓] Ya

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan pengalaman Anda mengerjakan tugas ini.

Pertanyaan 1

Dari seluruh *assertion* yang Anda tulis, mana yang paling berguna untuk menemukan kesalahan pada API Anda? Berikan alasannya.

Penjelasan:

Menurut saya assertion yang paling berguna Adalah “Data baru memiliki id dan nama sesuai kiriman” pada request Post Tambah Alumni. Assertion ini menyimpan sama persis sesuai dengan apa yang dikirimkan dengan membuat data baru dan id otomatis.

Assertion ini terbukti berguna karena pernah terjadi kesalahan pada saat menjalankan Post Tambah mahasiswa yang mengakibatkan server error 400 karena perbedaan nama dengan assertion yang membuat test gagal.

Dari kegagalan tersebut saya langsung dapat mengetahui kesalahan nya ada pada body yang dikirim, bukan pada fungsi API, lalu memperbaikinya sampai status 201 dan nama pada respons sesuai dengan yang dikirimkan.

Pertanyaan 2

Apa perbedaan yang Anda rasakan antara menguji API secara manual dan memakai Collection Runner? Bagian mana yang paling sulit saat menulis test script, dan bagaimana Anda mengatasinya?

Penjelasan:

Menurut saya pengujian API secara manual lebih mudah untuk mengetahui error nya karena dapat melihat pada bagian mana yang terjadi error melalui respons body dan test result. Kurang nya hanya proses nya lebih lama karena harus di uji satu persatu setiap request. Sedangkan memakai Collection Runner seluruh request dijalankan berurutan sekaligus dalam waktu kurang dari dua detik dan setiap assertion diperiksa otomatis sehingga hasilnya langsung terlihat sebagai PASS dan FAIL. Runner juga bisa diulang dengan hasil yang konsisten setiap kali kode API berubah.

Bagian tersulit adalah menjaga agar assertion sesuai dengan data yang dikirim dan menyambungkan antar request. Pada awalnya, nama di assertion berbeda dengan body request sehingga test gagal dan server membalas status 400. Untuk mengatasinya dengan membaca pesan kegagalan di hasil Runner, memeriksa body yang dikirim, lalu menyamakannya dengan nilai yang diharapkan.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [...] sudah diganti dengan data API saya	Ya
2	Setiap endpoint API saya ada di Collection	Ya
3	Alamat memakai {{base_url}}, bukan ditulis langsung	Ya
4	Setiap request punya assertion status code dan struktur JSON	Ya
5	Nama setiap test berupa kalimat yang menyatakan harapan	Ya
6	POST menyimpan ID dengan pemeriksaan status, dan request berikutnya memakainya	Ya
7	Server dinyalakan ulang sebelum Runner dijalankan	Ya
8	Screenshot Runner menampilkan 0 test gagal dan angkanya terbaca jelas	Ya
9	Berkas Environment tidak memuat kunci rahasia sungguhan	Ya
10	Berkas disimpan sebagai PDF dan dikumpulkan sesuai ketentuan LMS	Ya

Lampiran B Kriteria Penilaian

Kriteria berikut sesuai dengan slide materi Pertemuan 8. Bobot nilai mengikuti ketentuan dosen di LMS.

Kriteria	Deskripsi	Dinilai dari
Kelengkapan Postman Collection	Seluruh endpoint project akhir tercakup dalam satu Collection	Bab 1 dan Bab 2
Ketepatan automated test script	Assertion status code dan struktur response JSON tepat sasaran	Bab 3
Keberhasilan Collection Runner	Seluruh test lolos saat Collection dijalankan	Bab 4
Kejelasan dokumentasi hasil	Tangkapan layar ringkasan pengujian disertakan dengan jelas	Bab 4 dan seluruh dokumen